

Bioinformatics Lab2

BioInf 2025/2026 1st Semester

André Lamúrias

a.lamurias@fct.unl.pt

Problem 13 (BA5A)

- Coin change problem:
- Make change for M monetary units (e.g. cents)
- Use set of different coins (denomination) you can use (e.g. {1, 2, 5, 10, 20 and 50})
- You want to make change with the smallest number of coins
- We can assume we have as many coins of each type as needed
- Find the minimum number of coins (storing the actual coins to use would just add some bookkeeping to the algorithm).
- **What is the smallest number of coins we need to use to make change for M units given the list of coins S ?**

Problem 13 (BA5A)

- First approach: Greedy Algorithm

```
GREEDYCHANGE(money)  
  Change  $\leftarrow$  empty collection of coins  
  while money > 0  
    coin  $\leftarrow$  largest denomination that is less than or equal to money  
    add a coin with denomination coin to the collection of coins Change  
    money  $\leftarrow$  money – coin  
  return Change
```

- What's the issue?

Problem 13 (BA5A)

- Second approach: recurrence

```
RECURSIVECHANGE(money, COINS)
  if money = 0
    return 0
  minNumCoins  $\leftarrow \infty$ 
  for  $i \leftarrow 1$  to |COINS|
    if money  $\geq$  coin $i$ 
      numCoins  $\leftarrow$  RECURSIVECHANGE(money – coin $i$ , COINS)
      if numCoins + 1 < minNumCoins
        minNumCoins  $\leftarrow$  numCoins + 1
  return minNumCoins
```

- What's the issue?

Problem 1 (BA5A)

- Third approach: Bottom up (dynamic programming)

DPCHANGE(*money*, COINS)

MINNUMCOINS(0) $\leftarrow$ 0

for $m \leftarrow 1$ to *money*

MINNUMCOINS(m) $\leftarrow \infty$

for $i \leftarrow 1$ to |COINS|

if $m \geq \text{coin}_i$

if MINNUMCOINS($m - \text{coin}_i$) + 1 < MINNUMCOINS(m)

MINNUMCOINS(m) $\leftarrow$ MINNUMCOINS($m - \text{coin}_i$) + 1

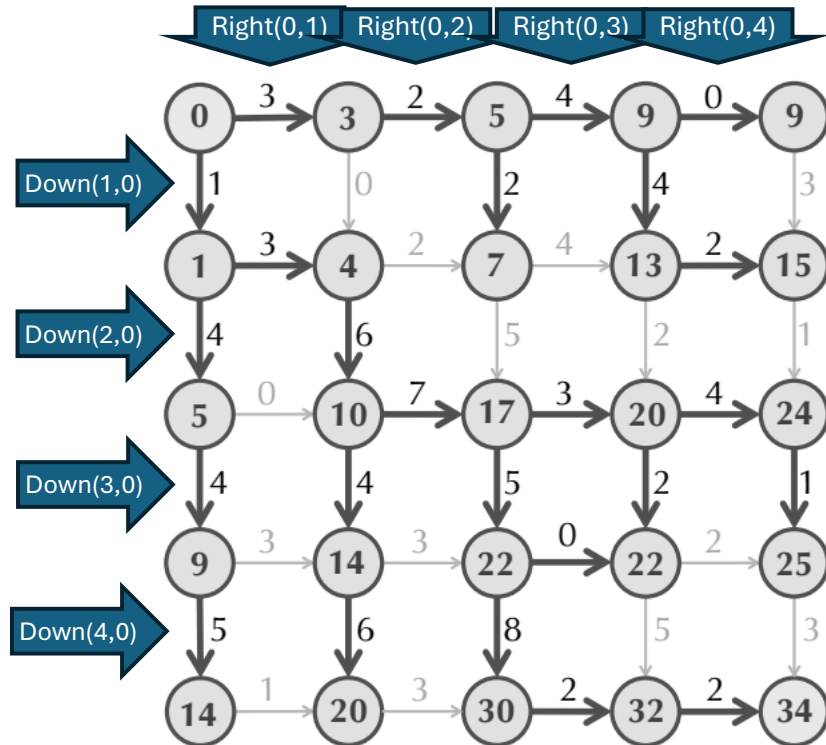
return MINNUMCOINS(*money*)

$$\text{MINNUMCOINS}(m) = \min \begin{cases} \text{MINNUMCOINS}(m - 5) + 1 \\ \text{MINNUMCOINS}(m - 4) + 1 \\ \text{MINNUMCOINS}(m - 1) + 1 \end{cases}$$

- What's the issue?

Problem 14 (BA5B)

- Length of a Longest Path in the Manhattan Tourist Problem



MANHATTANTOURIST($n, m, \text{Down}, \text{Right}$)

$s_{0,0} \leftarrow 0$

for $i \leftarrow 1$ to n

$s_{i,0} \leftarrow s_{i-1,0} + \text{down}_{i,0}$

for $j \leftarrow 1$ to m

$s_{0,j} \leftarrow s_{0,j-1} + \text{right}_{0,j}$

for $i \leftarrow 1$ to n

for $j \leftarrow 1$ to m

$s_{i,j} \leftarrow \max\{s_{i-1,j} + \text{down}_{i,j}, s_{i,j-1} + \text{right}_{i,j}\}$

return $s_{n,m}$

Problem 15 (BA5C)

- Longest Common Subsequence Problem
 - We need to retrieve the alignment (OutputLCS) after determining the longest path (LCSBacktrack)

```
OUTPUTLCS(Backtrack, v, i, j)
  if  $i = 0$  or  $j = 0$ 
    return
  if  $\text{Backtrack}_{i,j} = \downarrow$ 
    OUTPUTLCS(Backtrack, v,  $i - 1$ ,  $j$ )
  else if  $\text{Backtrack}_{i,j} = \rightarrow$ 
    OUTPUTLCS(Backtrack, v,  $i$ ,  $j - 1$ )
  else
    OUTPUTLCS(Backtrack, v,  $i - 1$ ,  $j - 1$ )
  output  $v_i$ 
```

```
LCSBACKTRACK(v, w)
  for  $i \leftarrow 0$  to  $|v|$ 
     $s_{i,0} \leftarrow 0$ 
  for  $j \leftarrow 0$  to  $|w|$ 
     $s_{0,j} \leftarrow 0$ 
  for  $i \leftarrow 1$  to  $|v|$ 
    for  $j \leftarrow 1$  to  $|w|$ 
       $s_{i,j} \leftarrow \max \begin{cases} s_{i-1,j} \\ s_{i,j-1} \\ s_{i-1,j-1} + 1, \text{ if } v_i = w_j \end{cases}$ 
      if  $s_{i,j} = s_{i-1,j}$ 
         $\text{Backtrack}_{i,j} \leftarrow \downarrow$ 
      else if  $s_{i,j} = s_{i,j-1}$ 
         $\text{Backtrack}_{i,j} \leftarrow \rightarrow$ 
      else if  $s_{i,j} = s_{i-1,j-1} + 1$  and  $v_i = w_j$ 
         $\text{Backtrack}_{i,j} \leftarrow \searrow$ 
  return Backtrack
```

Problem 15 (BA5C)

- Tips for LCSBacktrack
 - Draw the graph/table to understand the s and backtracking matrices
 - i, j means aligning position i of the first sequence with position j of the second sequence
 - $s_{i,j}$ means the longest path from 0,0 to i,j
 - $Backtrack_{i,j}$ means the direction that got us the longest path to i,j
 - There is no $Backtrack_{0,0}$
 - We don't want mismatches – add a gap instead
- If you use recursion for OutputLCS, in python:

```
import sys
sys.setrecursionlimit(10_000)
```


Problem 16 (BA5E)

- Global Alignment Problem with a scoring matrix (BLOSUM62) and gap penalty $\sigma = 5$



Global Alignment Problem:

Find a highest-scoring alignment of two strings as defined by a scoring matrix.

Input: Two strings and a scoring matrix *Score*.

Output: An alignment of the strings whose alignment score (as defined by *Score*) is maximized among all possible alignments of the strings.

$$s_{i,j} = \max \begin{cases} s_{i-1,j} - \sigma \\ s_{i,j-1} - \sigma \\ s_{i-1,j-1} + 1, & \text{if } v_i = w_j \\ s_{i-1,j-1} - \mu, & \text{if } v_i \neq w_j \end{cases}$$

	A	C	D	E	F	G	H	I	K	L	M	N	P	Q	R	S	T	V	W	Y
A	4	0	-2	-1	-2	0	-2	-1	-1	-1	-2	-1	-1	-1	1	0	0	-3	-2	
C	0	9	-3	-4	-2	-3	-3	-1	-3	-1	-1	-3	-3	-3	-1	-1	-1	-2	-2	
D	-2	-3	6	2	-3	-1	-1	-3	-1	-4	-3	1	-1	0	-2	0	-1	-3	-4	-3
E	-1	-4	2	5	-3	-2	0	-3	1	-3	-2	0	-1	2	0	0	-1	-2	-3	-2
F	-2	-2	-3	-3	6	-3	-1	0	-3	0	0	-3	-4	-3	-3	-2	-2	-1	1	3
G	0	-3	-1	-2	-3	6	-2	-4	-2	-4	-3	0	-2	-2	-2	0	-2	-3	-2	-3
H	-2	-3	-1	0	-1	-2	8	-3	-1	-3	-2	1	-2	0	0	-1	-2	-3	-2	2
I	-1	-1	-3	-3	0	-4	-3	4	-3	2	1	-3	-3	-3	-3	-2	-1	3	-3	-1
K	-1	-3	-1	1	-3	-2	-1	-3	5	-2	-1	0	-1	1	2	0	-1	-2	-3	-2
L	-1	-1	-4	-3	0	-4	-3	2	-2	4	2	-3	-3	-2	-2	-2	-1	1	-2	-1
M	-1	-1	-3	-2	0	-3	-2	1	-1	2	5	-2	-2	0	-1	-1	1	-1	-1	-1
N	-2	-3	1	0	-3	0	1	-3	0	-3	-2	6	-2	0	0	1	0	-3	-4	-2
P	-1	-3	-1	-1	-4	-2	-2	-3	-1	-3	-2	-2	7	-1	-2	-1	-1	-2	-4	-3
Q	-1	-3	0	2	-3	-2	0	-3	1	-2	0	0	-1	5	1	0	-1	-2	-2	-1
R	-1	-3	-2	0	-3	-2	0	-3	2	-2	-1	0	-2	1	5	-1	-1	-3	-3	-2
S	1	-1	0	0	-2	0	-1	-2	0	-2	-1	1	-1	0	-1	4	1	-2	-3	-2
T	0	-1	-1	-1	-2	-2	-2	-1	-1	-1	0	-1	-1	-1	1	5	0	-2	-2	
V	0	-1	-3	-2	-1	-3	-3	3	-2	1	1	-3	-2	-2	-3	-2	0	4	-3	-1
W	-3	-2	-4	-3	1	-2	-2	-3	-3	-2	-1	-4	-4	-2	-3	-3	-2	-3	11	2
Y	-2	-2	-3	-2	3	-3	2	-1	-2	-1	-1	-2	-3	-1	-2	-2	-2	-1	2	7

Problem 16 (BA5E)

- Tips:
 - You need to output the final score, as well as both sequences with gaps
 - All $(0,j)$ and $(i,0)$ positions should have $\text{score} = \text{gap} * i$ or $\text{gap} * j$

Problem 17 (BA5F)

- Local Alignment Problem
- We add another alternative:

$$s_{i,j} = \max \begin{cases} 0 \\ s_{i-1,j} + \text{Score}(v_i, -) \\ s_{i,j-1} + \text{Score}(-, w_j) \\ s_{i-1,j-1} + \text{Score}(v_i, w_j) \end{cases}$$

- Backtracking must start from highest values, and stop at a 0 value

Additional problems

- Problem 18 (BA5D): Generalized longest path in a edge-weighted DAG
- Problem 19 (BA5G): Compute the Edit Distance Between Two Strings
- Problem 20 (BA5J): Implement Affine Gap penalty